

Capstone II – Partner Onboarding & Config-Driven data Ingestion (JSLT)

Audience: Graduate Engineering Trainees (3–4 Backend, 1/2 Frontend per team)

Goal: Build a hands-on system mirroring production: partner onboarding in PostgreSQL, partner-specific **Loader Transformation Config** (Excel file defining column → datatype mappings), **backend-only JSLT generation** from that configuration (no UI wizard), and importing **Loader Data** (policy data Excel) into MongoDB after validation & transformation—with strong focus on code quality, ≥90% tests, resilience (circuit breaker, rate limit), and security.

Terminology

- **Loader Transformation Config:** An Excel file that lists the **column names and their data types** (and optional constraints). This drives **automatic JSLT generation**. Stored in MongoDB against a partnerId. There is **no UI** to build JSLT; it's generated server-side from this file.
- **Loader Data:** The **policy data** Excel to be validated and transformed using a selected Loader Transformation Config.

1) Scope

1. **Partner Onboarding (PostgreSQL)** – Create, edit limited fields, and list partners in a server-side paginated **data grid** with filter and multi-column sort.
2. **Config per Partner (MongoDB)** – Multiple **Loader Transformation Configs** per partner. Each Config → server generates **JSLT** + post-transform **JSON Schema** based on declared columns and datatypes.
3. **Loader Data Upload & Import (MongoDB)** – Given a Loader Transformation Config, upload a **Loader Data** Excel, validate columns/types/enums/formats, transform via generated JSLT to canonical JSON, and import into MongoDB keyed by partnerId and configId.

2) Architecture & Tech Choices

- **Backend:** Spring Boot 3 (Java 21), Gradle, Spring Data JPA (PostgreSQL), Spring Data MongoDB, Spring Validation, Spring Security, Resilience4j (circuit breaker, rate limiter, retry, bulkhead), Apache POI (Excel), **JSLT engine**, JSON Schema validation, Testcontainers, Micrometer.

- **Datastores:** PostgreSQL (partners), MongoDB (jslt_configs, imports, transformed_records), GridFS for uploaded files (Loader Transformation Configs and Loader Data).
- **Frontend:** Next.js 15 (App Router), React 19, TypeScript, Redux, React Hook Form, Zod, Onex-ui (Helix). **No UI for JSLT authoring FE** only selects config and uploads Loader Data files; provides previews/errors/history.
- **Observability:** Structured JSON logs (SLF4J), Loki; Grafana dashboards.
- **Quality Gates:** JUnit ≥90% backend; RTL for frontend; ESLint/TS strict.

3) Data Model

3.1 PostgreSQL – Sample schema for partners (This is minimal structure add more columns or foreign key table for additional information based on excels shared)

```
CREATE TABLE partners (
  id SERIAL PRIMARY KEY,
  partner_name VARCHAR(100) NOT NULL,
  type VARCHAR(50) NOT NULL CHECK (type IN ('Individual','Corporate')),
  email VARCHAR(100) NOT NULL UNIQUE,
  mobile VARCHAR(15) NOT NULL,
  contact_number VARCHAR(15) NOT NULL,
  pan VARCHAR(10) NOT NULL UNIQUE,
  gst VARCHAR(15),
  date_of_agreement DATE NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

3.2 MongoDB – sample Collections for storing JSLT configs

- **jslt_configs** (generated from **Loader Transformation Config** Excel; no manual UI)

```
{
  "_id": ObjectId,
  "partnerId": 123,
  "name": "MotorPolicy-v1",
  "policyType": "Motor|Health|...",
  "version": 1,
```

```

"description": "Generated from Loader Transformation Config Excel",
"columnMappings": [
  {"header": "Policy No", "key": "policyId", "type": "string", "required": true, "regex": "^[A-Z0-9-]{6,}$"},
  {"header": "Issue Date", "key": "issueDate", "type": "date", "format": "dd/MM/yyyy", "required": true},
  {"header": "Premium", "key": "premium", "type": "number", "min": 0}
],
"postTransformJsonSchema": { /* canonical model JSON Schema */ },
"jslt": "def parseDate(d) ... { ... }",
"sourceLoaderTransformationConfigMeta": {
  "fileId": ObjectId("..."),
  "filename": "motor_loader_transformation_config.xlsx",
  "sheetName": "Config",
  "sampleHash": "sha256:..."
},
"status": "draft|active",
"createdAt": ISODate(),
"updatedAt": ISODate()
}

```

- **Imports log** – One doc per **Loader Data** upload/validate/import.

```

{
  "_id": ObjectId,
  "partnerId": 123,
  "configId": ObjectId("..."),
  "fileId": ObjectId("..."),
  "filename": "motor_policies_sep.xlsx",
  "status": "validated|failed|imported",
  "rowCount": 1523,
  "errorRows": [{"row": 12, "errors": ["Issue Date invalid"]}],
  "transformedCount": 1510,
  "createdAt": ISODate(),
  "importedAt": ISODate()
}

```

- **transformed_records sample** – Canonical JSON per record.

```

{
  "_id": ObjectId,

```

```

"partnerId": 123,
"configId": ObjectId("..."),
"record": {"policyId": "AB-1234", "issueDate": "2025-09-05", "premium": 12000.5},
"sourceImportId": ObjectId("..."),
"createdAt": ISODate()
}

```

4) APIs

4.1 Partner Service (PostgreSQL)

- POST /api/partners → 201 {id,...}
- GET /api/partners/{id}
- PATCH /api/partners/{id} (editable subset)
- GET /api/partners?query=&type=&sort=created_at,DESC&page=0&size=20 (filter/sort/paginate)

4.2 Config Service (Mongo – backend-only generation)

- POST /api/partners/{partnerId}/loader-transformation-configs/upload (multipart **Loader Transformation Config** Excel)
 - Stores the raw Config in GridFS, parses **column→datatype** rows, and **generates JSLT + JSON Schema**.
 - Returns a **preview JSON** (columns, generated jslt, schema) and persists a **draft jslt_configs** doc.
- POST /api/partners/{partnerId}/configs/{configId}/activate → set status active.
- GET /api/partners/{partnerId}/configs / {configId} → list/read.

Loader Transformation Config Excel expected shape (example):

	Header Name	Canonical Key	Data Type	Required	Format/Regex	Enum Values
	Policy No	policyId	string	true	^[A-Z0-9-]{6,}\$	
	Issue Date	issueDate	date	true	dd/MM/yyyy	
	Premium	premium	number	false		
	Product	product	string	false		Car;Bike;CV

4.3 Ingestion Service (Loader Data: Upload/Validate/Import)

- POST /api/partners/{partnerId}/configs/{configId}/loader-data/upload (multipart **Loader Data** Excel) → 202 Accepted
 - Validates header presence/order, required columns, type/format/enum checks using the **generated** mappings.
 - **Empty column handling rule:** If a column cell is empty, ignore it and continue with the next column. If there are **five consecutive empty columns** following a non-empty one, treat that point as the **end of the valid data** (stop parsing further columns).
 - Transforms using stored **JSLT**; responds with {importId, status, errorRows[], transformedPreview:[...N]}.
- POST /api/imports/{importId}/commit → {status:"imported", transformedCount}
- GET /api/imports?... (filters) ; GET /api/imports/{importId} ; GET /api/transformed?...

Resilience & Limits

- Rate limit Loader Data & Loader Transformation Config uploads (e.g., 10/min per user).
- File size & row caps; circuit breakers on Mongo/DB/POI calls; retries for transient errors.

5) Frontend (Next.js 15 – App Router)

- **Partners:** data grid (search, filters, **multi-sort**, pagination), create/edit.
- **Configs:** list available configs per partner; show metadata (generated from Loader Transformation Config). **No JSLT editor UI.**
- **Loader Data:** upload Loader Data files to a chosen config; show validation errors table & transformed preview; commit import; history & detail views.
- All API will be routed via API end point of NextJS (BFF / Backend for front-end)
- **CSRF** for security only on front-end side (While fetching from API end point of NextJs)
- Use of **partial prerendering** of NextJs for smooth UI transitioning
- **Side panel** should be configurable based on JSON structure

6) Backend JSLT Generation Logic (from Loader Transformation Config)

1. **Parse Loader Transformation Config Excel** (sheet Config by default): iterate rows after header; build columnMappings with {header, key, type, required, format/regex, enums[]}

2. **Generate JSLT:**

- a. Date helper(s) per distinct format → ISO yyyy-MM-dd.
- b. Numeric coercion (remove commas, cast); boolean normalization (Y/N,1/0, true/false).
- c. Direct header→key mapping with null/trim guards.

3. **Generate Post-transform JSON Schema** from the columnMappings types (and min/max/enums if provided).

4. **Persist** draft config with sourceLoaderTransformationConfigMeta (fileId, hash). Optionally compute **drift warnings** on later Loader Data uploads if headers/types mismatch.

Example JSLT template:

```
def trim(s) : if (s == null) null else string(s)
```

```
def parse_ddMMyyyy(d) : if (d==null) null else string(.date:format(.date:parse(d, "dd/MM/yyyy"), "yyyy-MM-dd"))
```

```
def to_number(x) : if (x==null) null else if (is_number(x)) x else number(replace(string(x), ",", ""))
```

```
{  
  "policyId": trim(['Policy No']),  
  "issueDate": parse_ddMMyyyy(['Issue Date']),  
  "premium": to_number(['Premium']),  
  "product": trim(['Product'])  
}
```

6.1 Loader Data Column-Parsing Heuristics (Sparse/Trailing Empty Columns)

To mirror production behavior on ragged spreadsheets:

- **Skip isolated empty cells:** If a column is empty for a given row, **ignore it** and continue with the next column.
- **End-of-row detection:** After encountering any non-empty cell in a row, if there are **5 consecutive empty cells**, treat that as **end of data** for that row. Remaining cells are ignored.
- **Header alignment:** Validation still checks that all **declared headers** from the Loader Transformation Config appear in the sheet. Runtime sparsity only affects **row parsing**, not **schema presence**.

- **Metrics & logging:** Track trailingEmptyStreakMax, skippedEmptyCells, and per-column null rates; surface in the import summary for QA.
- **Config knobs** (*optional*): emptyStreakEndOfRow=5 (default), maxSkipsPerRow (sanity cap, e.g., 100), and treatCompletelyBlankRowAs: skip|error (default skip).

Algorithm sketch (row-level):

1. Scan columns left→right.
2. Record the index of the first non-empty cell (firstDataIdx).
3. Maintain consecutiveEmpty counter **only after** firstDataIdx is set.
4. If consecutiveEmpty == emptyStreakEndOfRow (5), **break** the scan (end of row).
5. For non-empty cells, reset consecutiveEmpty to 0 and map by header→key.

Validation note: Required fields are enforced **after** parsing—i.e., if a required field is missing/empty in the parsed slice, a validation error is emitted for that row.

7) Security, Reliability, Observability

- **Security:** Spring Security; server-side validation (Hibernate Validator + custom); file type sniffing; size/timeouts; antivirus stub; audit logs; secrets via env; no PII in logs.
- **Resilience:** Resilience4j rate limiter on upload; circuit breakers on DB/Mongo; retry/backoff; bulkhead for parse/transform worker pool.
- **Observability:** Micrometer timers/counters (rows parsed, error %, import duration); readiness/liveness; structured logs with traceId/spanId; Grafana dashboards.

8) Testing Strategy (≥90%)

- **Backend:** unit (parsers, JSLT gen, validators, **sparse-column heuristics**), slice (@DataJpaTest, @DataMongoTest), integration (Loader Transformation Config upload → config draft; Loader Data upload → validate → commit via Testcontainers), contract (OpenAPI), negative cases (bad types/enum/format, drift, **end-of-row detection with 5 empties**), performance (large Loader Data files).
- **Frontend:** unit (forms/table utils), component (grids, upload UI), E2E (Loader Data upload → preview → commit, history), a11y.
- **CI Gates:** JaCoCo ≥90% required; ESLint/TS strict;

9) Team Structure & Ownership) Team Structure & Ownership

Backend

- **B1 – Partner Service** (Postgres CRUD + list/sort/filter; audit; tests).
- **B2 – Config Service** (Loader Transformation Config upload → parse → **JSLT & Schema generation**; activation; drift detection; tests).
- **B3 – Ingestion Service** (Loader Data upload→validate→transform→commit; GridFS; thresholds; resilience; metrics; tests).
- **B4 – Cross-cutting** (security, audit, observability, CI/CD, gates).

Frontend

- **F1 – Partners & Configs UI** (partners grid/forms; config lists & metadata;).
- **F2 – Loader Data UI** (upload, validation preview, commit, history/detail;).

10) Acceptance Criteria (excerpt)

- Partners list search, filters, **multi-column sort**, server pagination ($P95 \leq 200\text{ms}$ @ page size 20).
- Uploading a **Loader Transformation Config** generates a draft config with JSLT & JSON Schema; activation flow present; no JSLT UI.
- Loader Data validation for 10k rows completes under 90s locally with progress feedback.
- **Sparse columns supported**: per-row isolated empties are skipped; **5 consecutive empties after first data** marks end-of-row.
- Import blocks when error rows exceed threshold; partial errors are reportable & downloadable.
- Mutations audited; coverage $\geq 90\%$; Grafana shows key ingestion metrics.

11) Deliverables for Trainees

- This spec (doc), OpenAPI YAML with examples.
- Seed SQL (partners) + seed Mongo (example jslt_configs).
- Sample **Loader Transformation Config** Excel and **Loader Data** Excel files.
- Docker Compose: Postgres, Mongo, Prometheus, Grafana, services.
- CI templates with coverage gates and E2E; PR checklist & rubric.

Partner Registration

Partner Name

Partner Type

Email

Contact Number

PAN Number

GST Number

Contact Address

Date of Agreement

Submit

Edit Partner

Partner Name

Partner Type

Email

Contact Number

PAN

Contact Address

Date of Agreement

Edit

Partner List

View onboarded partners and their details.

Partner Name	Type	Email	Contact Number	Date of Agreement	PAN
John Smith	Individual	john.smith@example.com	9876543210	2024-04-10	Edit
Acme Corporation	Corporate	contact@acme.exanple	9986776655	2024-04-08	Edit
Jane Doe	Individual	jane.doe@example.com	9123456789	2024-04-05	Edit
Global Industries	Corporate	info@global.example	9234557890	2024-04-02	Edit

- New business
- Partner
- Policy plan setup
- Loader setup
- Derivation config
- MPH onboarding
- Report & downloads
- New business
- FR documents
- MIF documents
- Claims
- FR docs (Claims)
- COIs
- Pre issuance cancellation
- Fund management
- Claims
- User management
- My Team

Loader transformation config

CLIENT MIGRATEDCLIENT MIG...

MFI | Standard template

Reset

Choose a file or drag and drop here
XLSX (Max 25 MB)

Browse

Loaders

Date	Loader Id	Template Name	Loader Type	Uploaded by	Actions
09/09/2025, 11:59	68bfc95475eeee231e88c2d3	Loader Transformation - Sp...	MFI Standard template	sudamaj	Download
03/09/2025, 21:58	68b86c9016f25b1fd18ba627	Loader Transformation - Sp...	MFI Standard template	sudamaj	Download
03/09/2025, 21:00	68b85f1f16f25b1fd18ba623	Loader Transformation - Sp...	MFI Standard template	sudamaj	Download

- New business
- Partner
- Policy plan setup
- Loader setup
- Derivation config
- MPH onboarding
- Report & downloads
- Fund management
- Claims
- User management
- My Team
- Dashboard

Loader transformation config

ASIRVAD MICRO FINANCE LL...

MFI | SHG

Reset

Choose a file or drag and drop here
XLSX (Max 25 MB)

Browse

Loaders

Date	Loader Id	Template Name	Loader Type	Uploaded by
04/09/2025, 15:57	68b9697bbe48cd17a69ffd18	Loader Transformation Easy...	MFI SHG	c.kalpesh
07/08/2025, 17:09	689490593a7e7174dacd1562	Loader Transformation Easy...	MFI SHG	tho_cor26
07/08/2025, 17:02	68948ee23a7e7174dacd1560	Loader Transformation Easy...	MFI SHG	tho_cor26
07/08/2025, 17:02	68948eb74b8372343fd60ca7	Loader Transformation Easy...	MFI SHG	tho_cor26

Upload

New business

Partner

Loader Type

Choose a file or drag and drop here
XLS, XLSX, XLSB, XLSM, CSV (Max 100 MB)

Browse

Upload maximum 5 files at a time.